SAN assignment - Dimensionality reduction

Author : Nguyen Thu Uyen Sarah

Email : nguyet79@fel.cvut.cz

In [45]:
```
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
from scipy.sparse.csgraph import shortest_path
from scipy.spatial import voronoi_plot_2d, Voronoi, cKDTree
from sklearn.decomposition import PCA
from sklearn.manifold import MDS, TSNE
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
```

Introduction

The goal of this tutorial is to get familiar with some basic methods for dimensionality reduction, complete you own implementation of the **Isomap algorithm** (in cooperation with *Multidimensional scaling*), experiment with its parameters and compare with other techniques of dimensionality reduction (**PCA**, **t-SNE**).

Your task is to complete prepared Isomap implementation and inspect the mehods behavior. See **all** sections with `# TODO:` . Submit your solution consisting of both this modified notebook file and the exported PDF/HTML document as an archive to the courseware BRUTE upload system for the SAN course. The deadline is specified there.

Background

The data you will be working with are vector representations of words in a latent (unknown) high-dimensional space. This representation of words, also know as word embedding, differs from standard bag-of-words (BoW, TFIDF, etc.) representations in that the meaning of the words is distributed across all the dimensions. Generally speaking, the word embedding algorithms seek to learn a mapping projecting the original BoW representation (simple word index into a given vocabulary) into a lower-dimensional (but still too high for our case)

continuous vector-space, based on their distributional properties observed in some raw text corpus. This distributional semantics approach to word representations is based on the basic idea that linguistic items with similar distributions typically have similar meanings, i.e. words that often appear in a similar context (words that surround them) tend to have similar (vector) representations.

Speciffically, the data you are presented with are vector representations coming from the most popular algorithm for word embedding known as word2vec [1] by Tomas Mikolov (VUT-Brno alumni). *Word2vec* is a (shallow) neural model learning the projection of BoW word representations into a latent space by the means of gradient descend. Your task is to further reduce the dimensionality of the word representations to get a visual insight into what has been learned.

[1]: Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pages 3111-3119, 2013.

Data

You are given 300-dimensional word2vec vector embeddings in the file *data.csv* with corresponding word labels in *labels.txt* for each line. Each of these words comes from one of 10 selected classes of synonyms, which can be recognized (and depicted) w.r.t. labels denoted in the file *colors.csv*

In [46]:
```
def plot_points(X, labels, colors, title):
    voronoi = Voronoi(X)
    fig, ax = plt.subplots(figsize=(10,8), dpi=128)
    voronoi_plot_2d(voronoi, ax, show_points=False, show_vertices=False)
    ax.scatter(X[:, 0], X[:, 1], c=colors, s=8, cmap=mpl.colormaps["tab10"])
    for point, label in zip(X, labels):
        ax.text(point[0], point[1], label[0], va = "top", ha = "center", fonts
    plt.title(title)
    plt.show()
```

In [47]:
```
def test_classification(X, colors, n_folds=10):
    accuracy = []
    skf = KFold(n_splits=n_folds, shuffle=True, random_state=0)
    for train_idxs, test_idxs in skf.split(X, colors):
      tree_model = DecisionTreeClassifier().fit(X[train_idxs], colors[train_id
      prediction = tree_model.predict(X[test_idxs])
      accuracy.append(np.mean(prediction == colors[test_idxs, 0]))
    return accuracy
```

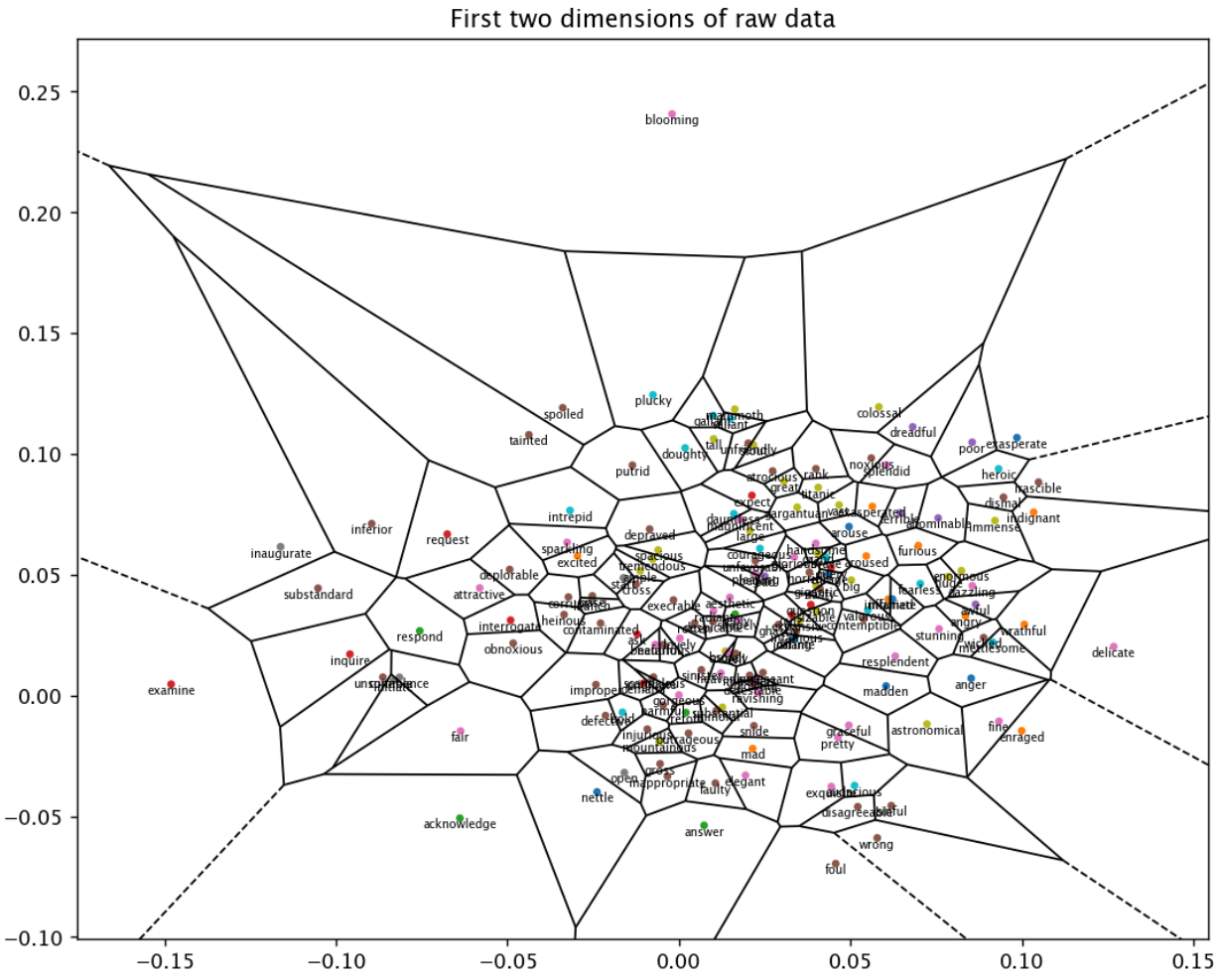
Tasks

1. **Load the dataset of 165 words**, each represented as a 300-dimensional vector. Each word is assigned to one of 10 clusters.

In [48]:
```
#Load the dataset of 165 words
my_data = pd.read_csv("data.csv", header=None).values
my_labels = pd.read_csv("labels.txt", header=None).values
my_colors = pd.read_csv("colors.csv", header=None).values
```

The data is in the matrix `mydata`, cluster assignment in `mycolors` and the actual words (useful for visualization) in `mylabels`. You can plot the data by using only the first 2 dimensions.

In [49]:
```
plot_points(my_data[:, [0,1]], my_labels, my_colors, title="First two dimensic
```



2. **Implement ISO-MAP dimensionality reduction procedure**.

- Use *k*-NN method to construct the neighborhood graph (sparse matrix).
    - For simplicity, you can use `cKDTree` method available in `sklearn` package.
- Compute shortest-paths (geodesic) matrix using your favourite algorithm.
    - Tip: you can use `shortest_path` method from scipy.
- Project the geodesic distance matrix into 2D space with (Classical) Multidimensional Scaling (`MDS` method from sklearn).
- Challenge: you may simply use PCA to do the same, but be careful to account for a proper normalization (centering) of the geodesic (kernel) matrix (see Kernel PCA for details).

An expected result (for $k = 5$) should look similar (not necessarily exactly the same) to following

Example output

In [50]:
```
def isomap(data, k):
    """
    Your function computing the ISOMAP embedding for data.

    :param data: numpy array (m, d) where m is number of samples and d is the
    :param k: number of neighbours to consider for constructing the neighborhc
    :return: data embedded into R^2 acc. to the ISOMAP algorithm
    """

    # 1. Determine the neighbors of each point (k-NN)
    tree = cKDTree(data)
    knn_distances, knn_indices = tree.query(data, k=k+1)

    # Remove the first column (self-reference)
    knn_indices = knn_indices[:, 1:]
    knn_distances = knn_distances[:, 1:]

    # 2. Construct the neighborhood graph.
    # Each point is connected to other if it is a K nearest neighbor.
    # Edge length equal to Euclidean distance.
    num_vertices = data.shape[0]
    num_neighbors = k

    adj_mat = np.full((num_vertices, num_vertices), np.inf)
    np.fill_diagonal(adj_mat, 0)

    for i in range(num_vertices):
        for j in range(num_neighbors):
            node_from = i
            node_to = knn_indices[i, j]
            dist = knn_distances[i, j]
```

```
            # undirected graph
            adj_mat[node_from, node_to] = dist
            adj_mat[node_to, node_from] = dist

    # 3. Compute the shortest path between two nodes (Floyd-Warshall algorithm
    dist_matrix = adj_mat.copy()

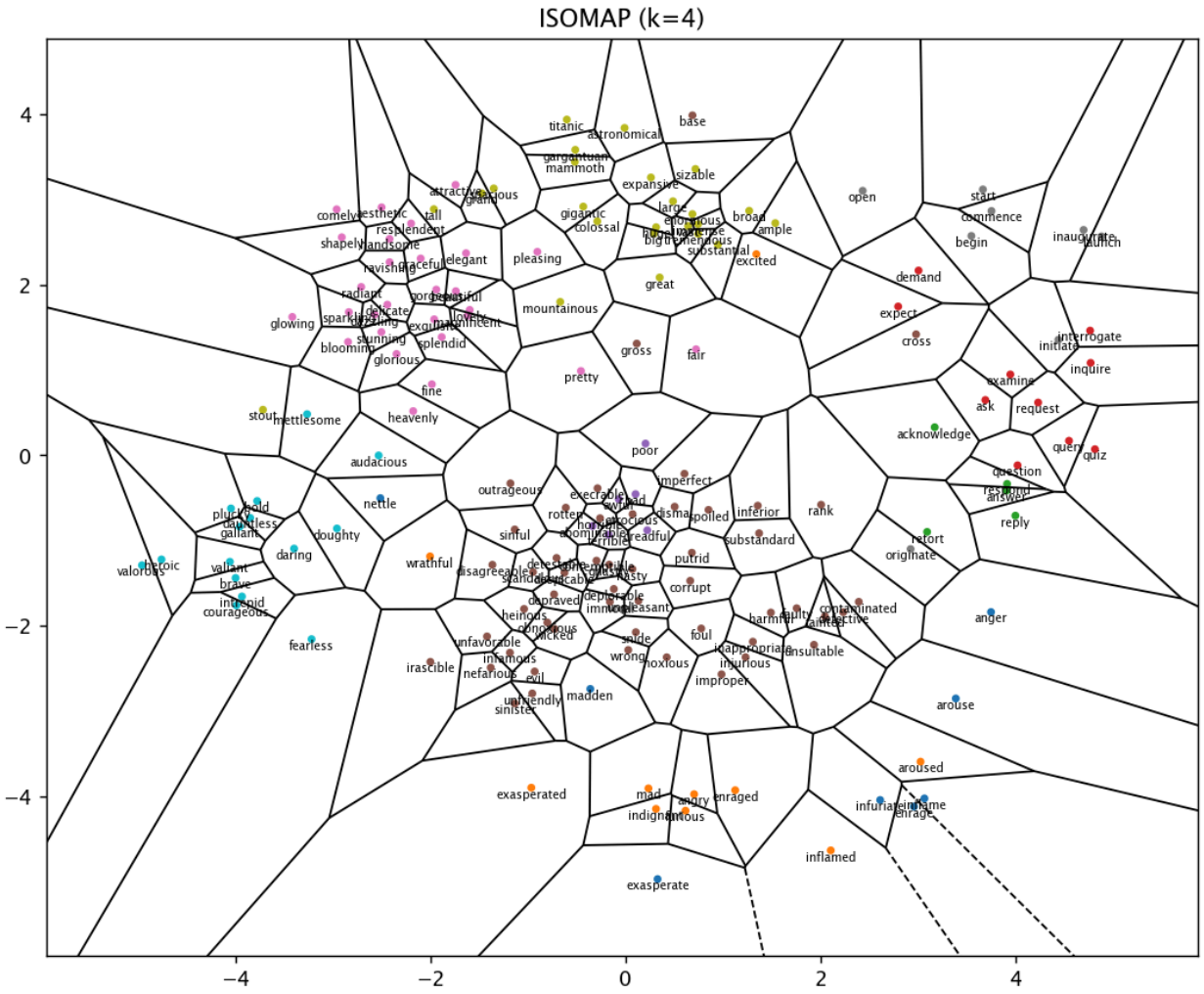
    for k_idx in range(num_vertices):
        for i in range(num_vertices):
            for j in range(num_vertices):
                dist_matrix[i, j] = min(dist_matrix[i, j],
                                        dist_matrix[i, k_idx] + dist_matrix[k_i

    # 4. (classical) multidimensional scaling
    mds = MDS(n_components=2, dissimilarity='precomputed', random_state=42)
    projection = mds.fit_transform(dist_matrix)

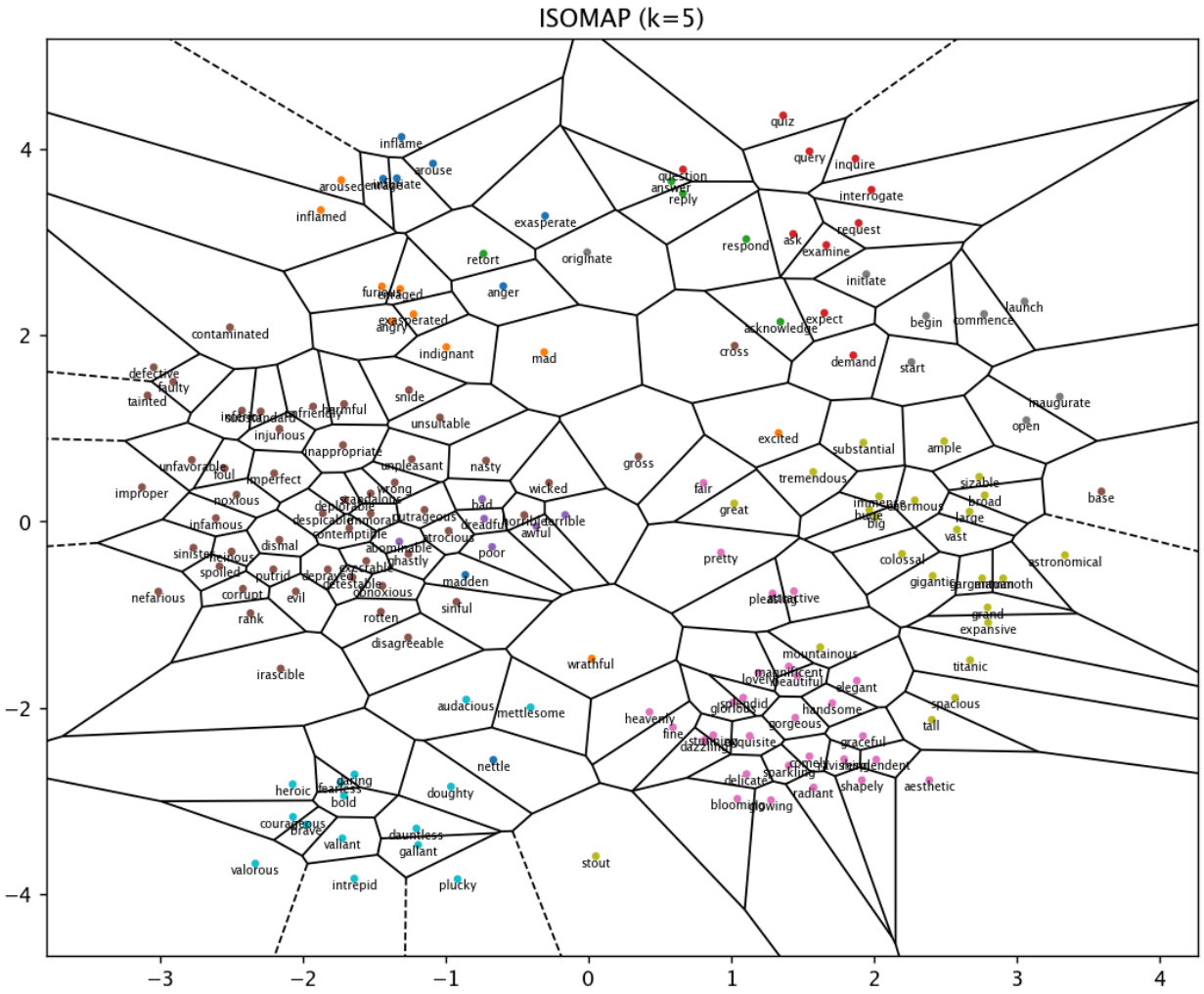
    return projection


for k in range(4, 10):
    projection_isomap = isomap(my_data, k=k)
    plot_points(projection_isomap, my_labels, my_colors, title=f"ISOMAP (k={k}
```

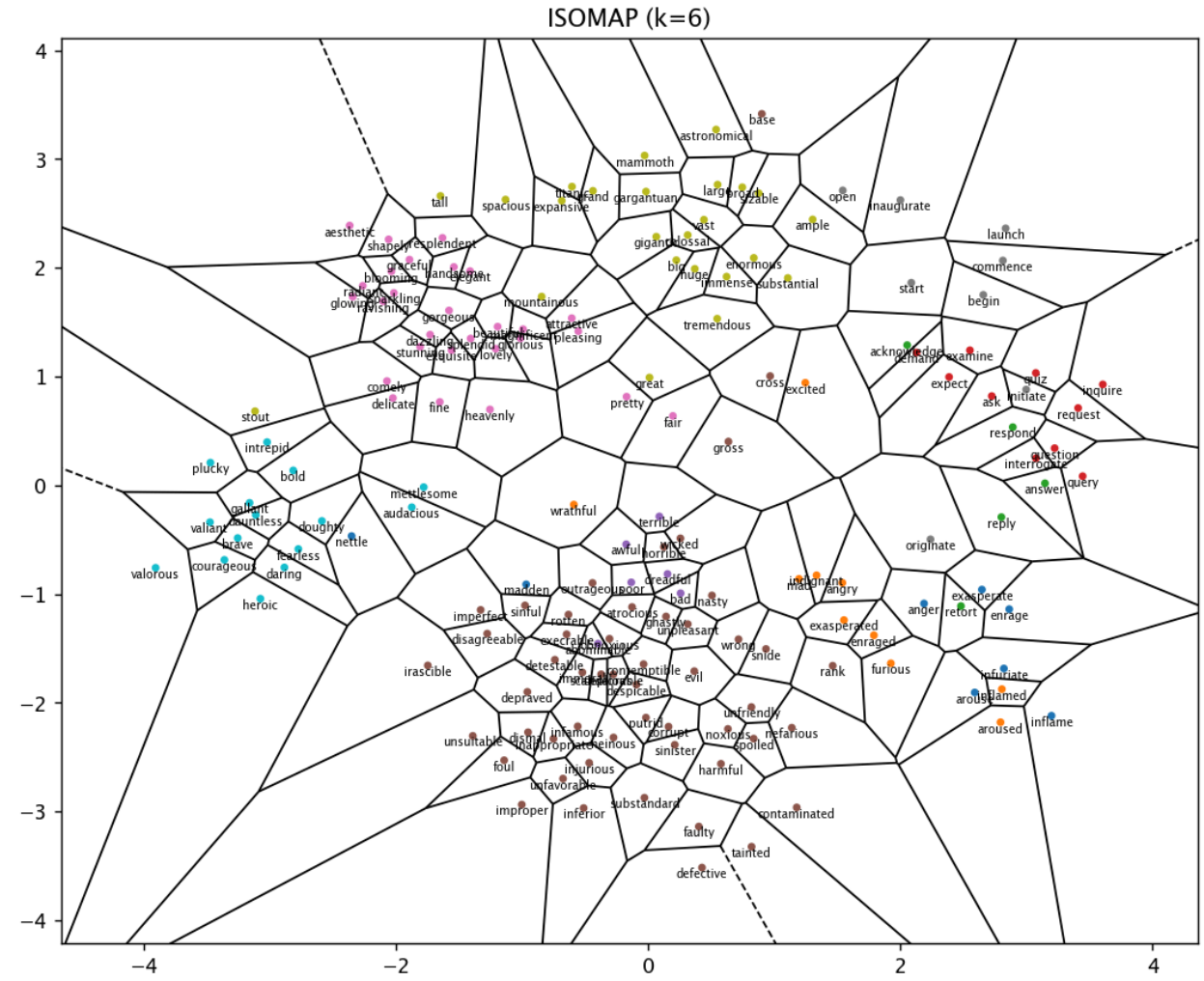
```
/Users/thuuyen/School/SAN/assignment4/.venv/lib/python3.13/site-packages/sklear
n/manifold/_mds.py:677: FutureWarning: The default value of `n_init` will chang
e from 4 to 1 in 1.9.
  warnings.warn(
```



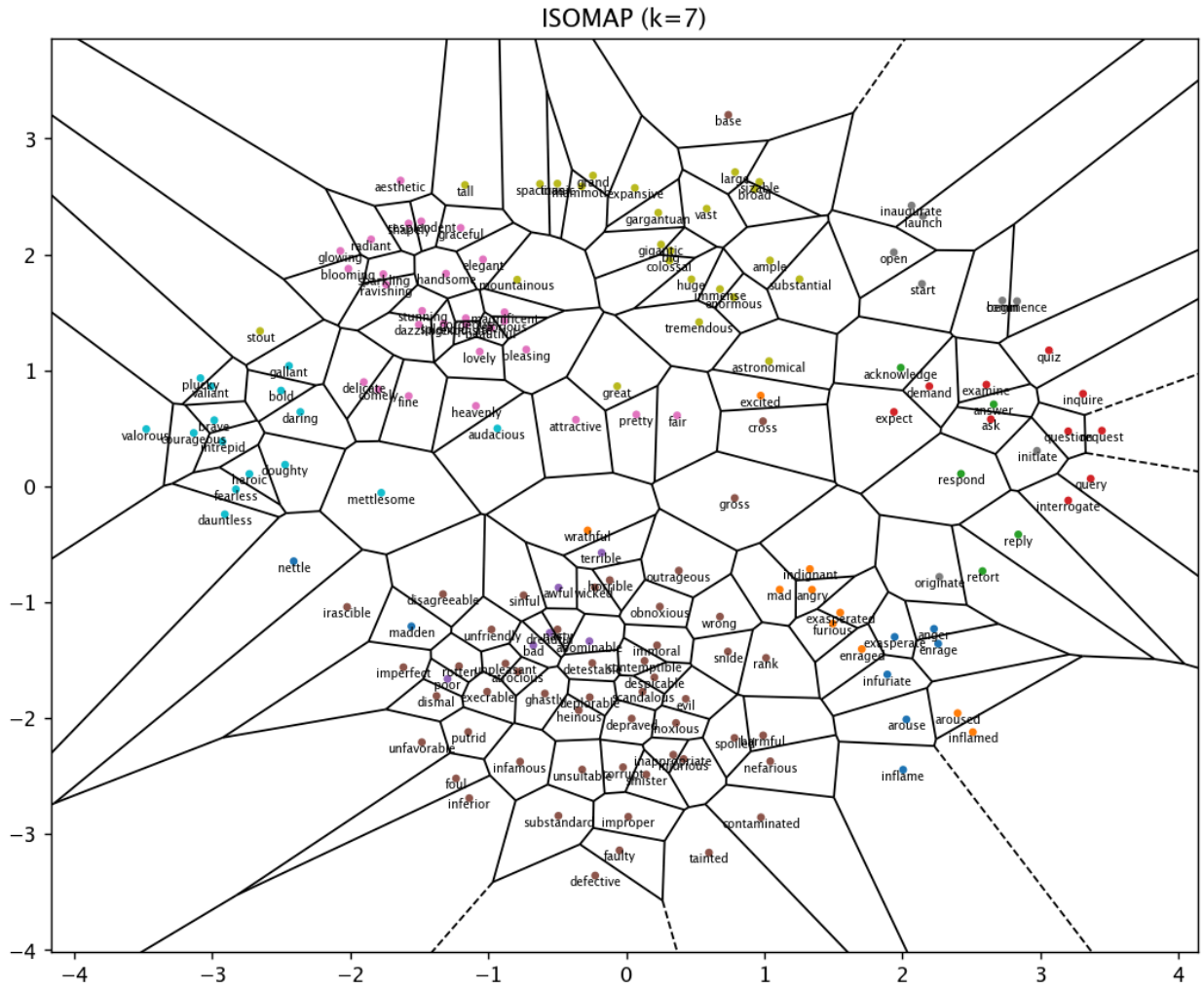

```
/Users/thuuyen/School/SAN/assignment4/.venv/lib/python3.13/site-packages/sklearn/manifold/_mds.py:677: FutureWarning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```



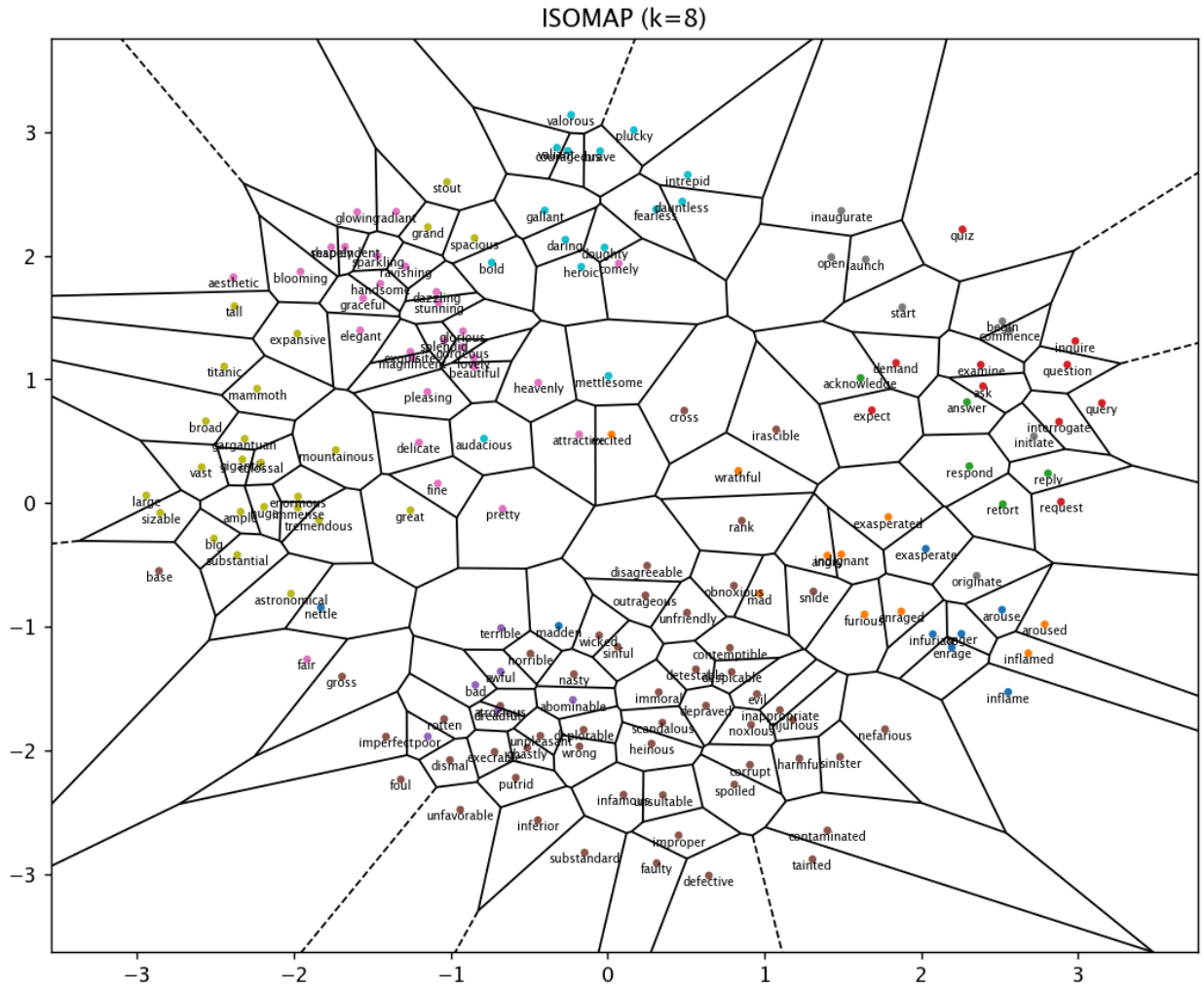

```
/Users/thuuyen/School/SAN/assignment4/.venv/lib/python3.13/site-packages/sklearn/manifold/_mds.py:677: FutureWarning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```



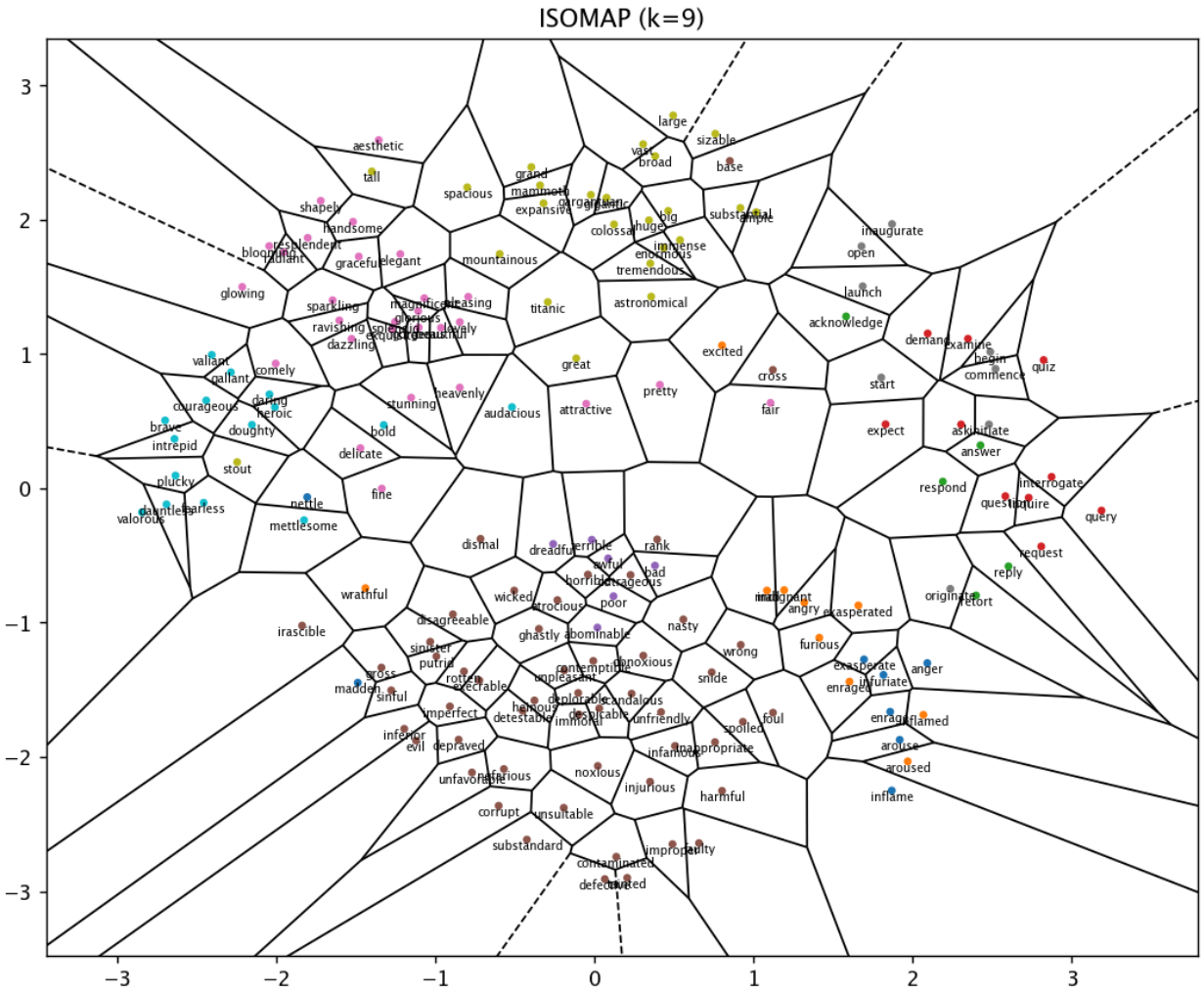

```
/Users/thuuyen/School/SAN/assignment4/.venv/lib/python3.13/site-packages/sklearn/manifold/_mds.py:677: FutureWarning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```




```
/Users/thuuyen/School/SAN/assignment4/.venv/lib/python3.13/site-packages/sklearn/manifold/_mds.py:677: FutureWarning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```



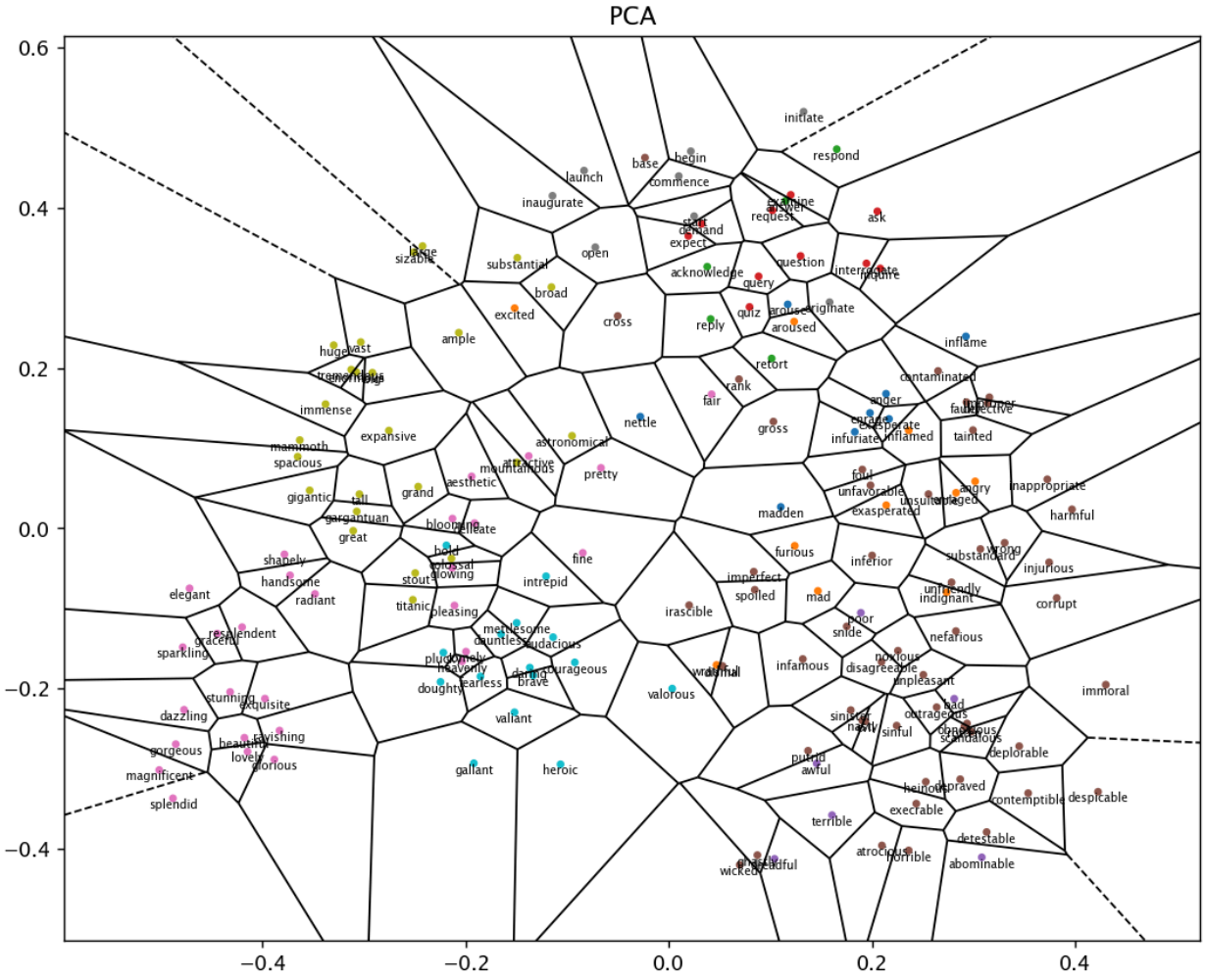

```
/Users/thuuyen/School/SAN/assignment4/.venv/lib/python3.13/site-packages/sklearn/manifold/_mds.py:677: FutureWarning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```

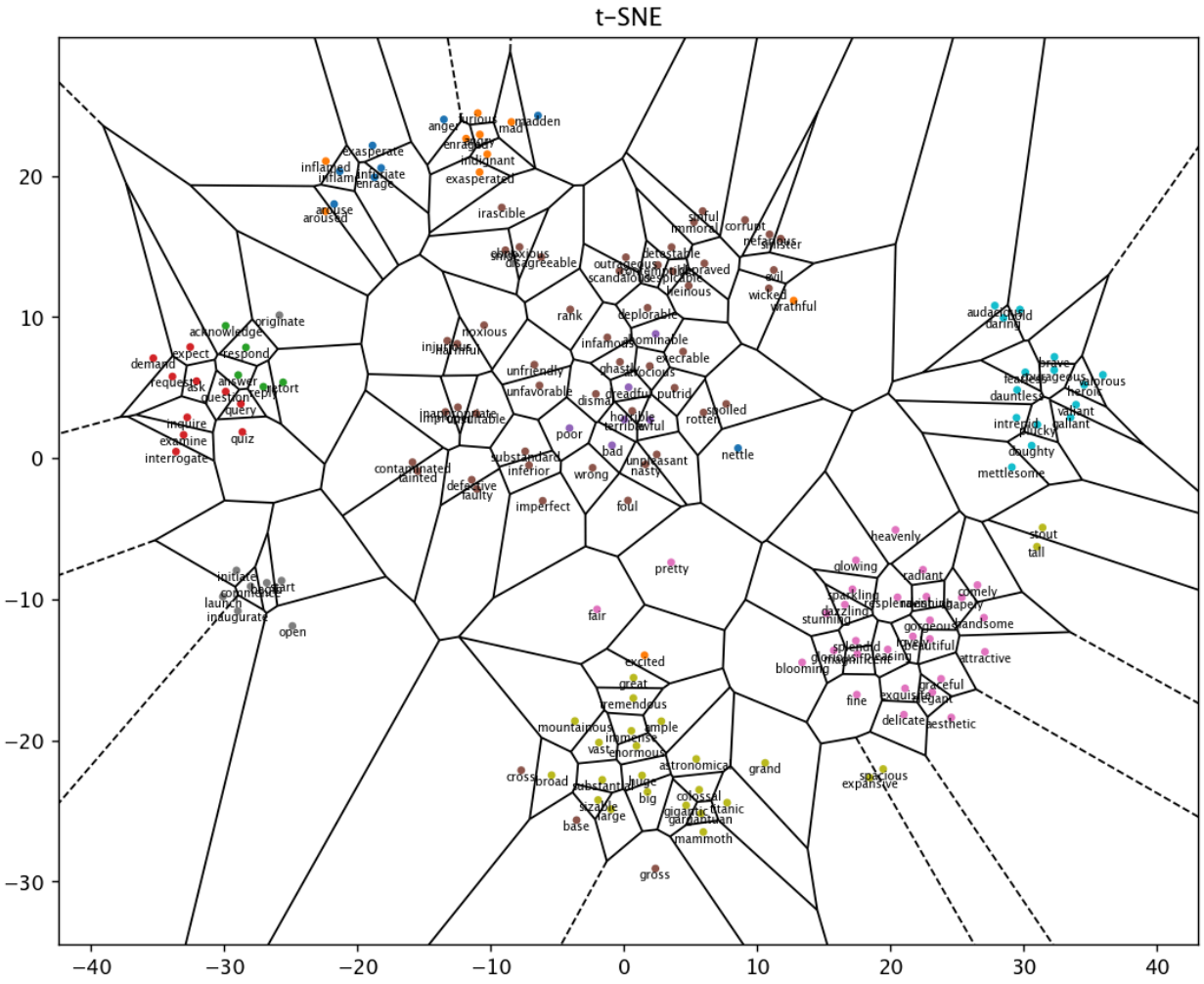


3. **Visually compare PCA, ISOMAP and t-SNE** by plotting the word2vec data, embedded into 2D using the `Plotpoints` function. Try finding the optimal *k* value for ISOMAP's nearest neighbour.

In [51]:
```
#Principal component analysis
scaled_data = StandardScaler().fit_transform(my_data)
projection_pca = PCA(2).fit_transform(my_data)
plot_points(projection_pca[:, [0, 1]], my_labels, my_colors, title="PCA")

#t-SNE (T-Distributed Stochastic Neighbor Embedding)
projection_tsne = TSNE(n_components=2, perplexity=10, init="random", learning_
plot_points(projection_tsne, my_labels, my_colors, title="t-SNE")
```


PCA
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.4
−0.2
0.0
0.2
0.4
initiate
respond
launch
base
begin
commence
inaugurate
request
ask
demand
expect
open
substantial
question
acknowledge
query
broad
excited
cross
reply
quiz
arouse
aroused
ample
inflame
retort
rank
fair
contaminated
nettle
gross
infuriate
inflamed
tainted
astronomical
pretty
aesthetic
grand
gigantic
tall
gargantuan
great
expansive
immense
mammoth
spacious
madden
furious
inferior
substandard
inappropriate
harmful
injurious
corrupt
irascible
spoiled
mad
poor
snide
nefarious
infamous
disagreeable
unpleasant
immoral
valorous
valiant
gallant
heroic
sinful
deplorable
despicable
contemptible
execrable
terrible
detestable
atrocious
horrible
abominable
wicked
elegant
handsome
radiant
shapely
titanic
pleasing
intrepid
fine
stout
bold
colossal
glowing
sparkling
resplendent
graceful
dazzling
stunning
exquisite
gorgeous
magnificent
splendid
lovely
glorious
heavenly
courageous
brave
daring
doughty
fearless
audacious
dauntless
meritsome



4. **Observe the effect of dimensionality reduction on a classiffication algorithm**. The supporting code in a function `test_classification` performs training and testing of classification trees and gives the classification accuracy (percentage of correctly classified samples) on individual cross-validation test splits as its result. Compare the accuracy of prediction on plain data, PCA, ISOMAP and t-SNE.

In [52]:
```
#classify ISOMAP
acc_isomap = test_classification(projection_isomap, my_colors, 50)

#classify PCA
acc_pca = test_classification(projection_pca, my_colors, 50)

#classify t-SNE
acc_tsne = test_classification(projection_tsne, my_colors, 50)

#PLOT results
print(f"ISOMAP ACC: {np.mean(acc_isomap):>10,.3f}")
```

```
print(f"PCA ACC: {np.mean(acc_pca):>13,.3f}")
print(f"t-SNE ACC: {np.mean(acc_tsne):>11,.3f}")
```

```
ISOMAP ACC:      0.652
PCA ACC:         0.565
t-SNE ACC:       0.810
```

Your comment. What did you observe?

The results show that t-SNE achieved the highest accuracy, while PCA and Isomap performed similarly to each other.

Theoretical Comparison

Isomap and t-SNE are both distance-based methods that attempt to preserve local neighborhood structures when projecting data to lower dimensions. They aim to keep points that are close together in high-dimensional space close together in the reduced representation. In contrast, PCA takes a different approach by identifying a new subspace (typically 2D or 3D for visualization) that captures the maximum variance in the data, effectively retaining the most information about the original point positions.

Practical Considerations

For classification tasks, PCA tends to be more stable and reliable. Distance-based methods like t-SNE and Isomap can produce dramatically different embeddings when data points are added or removed, making them less suitable for classification purposes. However, standard PCA has limitations when dealing with non-linear manifolds, as it assumes linear relationships in the data. In such cases, distance-based methods may better capture the underlying structure despite their instability.

Feedback

Was some part of the notebook unclear, would any topic need more attention during the tutorials? If you want to leave us feedback on the assignment, we would be happy to hear it. Here is your space:

In [ ]: